Carga de los datos al clúster y próximos pasos

Grupo 13 · 12 de noviembre de 2026

1. Qué se hizo

Los archivos generados en Colab (carpeta **salida_v2**) se llevaron al clúster del curso. Se descargaron de Drive al computador de un integrante, se copiaron con **scp** a la carpeta local del grupo en el master (`/data/2026/uhadoop/projects/grupo13`) y desde allí se pasaron a HDFS (`/uhadoop2026/projects/grupo13`). Ambas carpetas se crearon siguiendo la convención de los otros grupos (grupo2, grupo4, grupo7, grupo8, grupo11), que se revisó antes con `ls`.

Pasos realizados:

- Se comprobó en el computador que los 7 archivos descargados tenían el tamaño esperado.
- Se creó la carpeta **grupo13** en el disco local del master y en HDFS.
- Se copiaron los archivos con `scp -P 220`. La transferencia terminó sin errores (unos 7 a 9 MB/s).
- Se comparó el tamaño en bytes de cada archivo en el clúster con el del computador: coincidieron los 7 exactamente.
- Se subieron a HDFS con `hdfs dfs -copyFromLocal` (replicación 3, el valor por defecto; el total es de unos 140 MB, muy por debajo de los 20 GB desde los que el profe recomienda bajarla a 1).
- Se verificó el número de líneas de los archivos clave directamente en HDFS (tabla siguiente).

2. Archivos cargados en HDFS

Archivo	Tamaño	Para qué sirve	Líneas en HDFS
ordenes_limpias.tsv.bz2	81,5 MB	Insumo principal de Pig: una fila por orden, 30 columnas, con cabecera. Comprimido en bz2, que Hadoop puede repartir entre máquinas.	1.856.039 (1.856.038 órdenes + cabecera) OK
ordenes_muestra.tsv	4,0 MB	10.000 órdenes al azar para desarrollar y probar los scripts de Pig.	10.001 OK
aristas.tsv	10,4 MB	Grafo organismo a proveedor, dirigido, sin pesos. Insumo de Giraph (PageRank).	476.703 OK
aristas_bidireccional.tsv	20,7 MB	Las mismas aristas en ambos sentidos (variante B de PageRank).	no verificado (se espera 953.406)
aristas_muestra.tsv	193 KB	Grafo construido solo con la muestra, para probar Giraph.	no verificado
nodos.tsv	4,0 MB	Nombre de cada organismo y proveedor, para ponerle nombre a los resultados.	76.766 (76.765 nodos + cabecera) OK
proveedor_rubro.tsv	16,2 MB	Monto neto por proveedor y rubro, para describir los perfiles.	no verificado

No se subió `ordenes_limpias.tsv` sin comprimir (774 MB), porque el archivo .bz2 contiene exactamente los mismos datos.

3. Copias locales: por qué todavía no se borran

El profe pide eliminar los datos locales del master una vez que están en HDFS, para no ocupar espacio. Las copias locales del grupo **todavía no se han borrado**, y hay una razón práctica.

Pig puede ejecutarse de dos maneras. En modo local (`pig -x local`) corre en una sola máquina y lee archivos del disco del master, no de HDFS. Es la forma rápida de comprobar que un script no tiene errores, usando **ordenes_muestra.tsv**, antes de lanzarlo sobre los 1,8 millones de órdenes en el clúster, donde cada error cuesta minutos de cola. Por eso conviene conservar la copia local al menos de la muestra hasta terminar esa prueba.

Cuando la prueba local esté lista, se borran todas las copias locales (solo las del grupo; el usuario *uhadoop* es compartido, así que no se usa `rm` sobre carpetas de otros):

```
rm /data/2026/uhadoop/projects/grupo13/{ordenes_limpias.tsv.bz2,ordenes_muestra.tsv,aristas.tsv,\
   aristas_muestra.tsv,aristas_bidireccional.tsv,nodos.tsv,proveedor_rubro.tsv}
```

Los archivos de HDFS, los del computador y los de Drive no se ven afectados. Si hiciera falta una copia local más adelante, se puede traer de nuevo desde HDFS con `hdfs dfs -copyToLocal`.

4. Cómo continuar: Pig

Objetivo (aprobado por el profe): rankings de proveedores por monto total, número de órdenes distintas y número de organismos distintos, más la concentración del gasto y la evolución mensual. **Archivos:** ordenes_limpias.tsv.bz2 (completo) y ordenes_muestra.tsv (pruebas).

Reglas que el script de Pig debe respetar:

- Filtrar la cabecera (`codigo != 'codigo'`) y quedarse solo con las órdenes con `usar == 1`.
- Sumar `monto_clp_limpio`, no `monto_clp`: la columna limpia deja vacías las órdenes sospechosas, simbólicas e inválidas (los vacíos se leen como null y se tratan como 0).
- Como el archivo ya tiene una fila por orden, el número de órdenes es un `COUNT` y el de organismos un `DISTINCT` de `cod_organismo` por proveedor.

Pasos, en orden:

- **Paso 1: llevar el script al master.** Se copia el .pig a la carpeta del grupo con `scp -P 220`.
- **Paso 2: prueba local con la muestra.** En la carpeta local del grupo:

```
pig -x local -param ENTRADA=ordenes_muestra.tsv -param SALIDA=pig_muestra g13_rankings.pig
cat pig_muestra/ranking_monto/part-*
```

La carpeta de salida no debe existir de antes; si hay que repetir la prueba, se cambia el nombre o se borra solo esa carpeta. Los resultados de la muestra se revisan a mano: que haya 30 columnas bien leídas, que no aparezca la cabecera como proveedor y que los montos tengan sentido.

- **Paso 3: ejecución completa en el clúster, dentro de screen**, para que no se corte si se cierra la conexión:

```
screen -S pig13
pig -param ENTRADA=/uhadoop2026/projects/grupo13/ordenes_limpias.tsv.bz2 \
    -param SALIDA=/uhadoop2026/projects/grupo13/pig_resultados g13_rankings.pig
# salir sin detener: Ctrl+A y luego D   |   volver: screen -d -r pig13   |   ver trabajos: mapred job -list
```

- **Paso 4: revisar resultados** con `hdfs dfs -cat .../pig_resultados/ranking_monto/part-* | head` y compararlos con los de control de Colab (`control_metricas_proveedor.csv` y `control_evolucion_mensual.csv`, en salida_v2): los totales por proveedor y por mes deben coincidir. Referencia: 75.600 proveedores, 1.853.390 órdenes usadas y 18,18 billones de pesos de monto limpio.
- **Paso 5: sacar los resultados** (son archivos pequeños) con `hdfs dfs -getmerge` o `-copyToLocal`, y bajarlos al computador con `scp -P 220` en sentido inverso, para usarlos en el informe y la presentación.

5. Cómo continuar: Giraph (PageRank)

Objetivo: calcular PageRank sobre el grafo organismo a proveedor y compararlo con el ranking por monto, para encontrar los dos perfiles que mencionó el profe: (1) alto monto y bajo PageRank, probablemente un contrato masivo único; (2) bajo monto y alto PageRank, proveedores que abastecen a organismos importantes. **Archivos:** aristas.tsv (dirigido), aristas_bidireccional.tsv, aristas_muestra.tsv y nodos.tsv.

- **Compilar el JAR.** Se usa el proyecto de Eclipse del Lab 8 (PageRank.java): con *build.xml* se genera **mdp-giraph.jar** en la carpeta *dist* y se copia al master. Eclipse solo hace falta para compilar.

- **Prueba de corrección** con el ejemplo del laboratorio (`pr-ex.tsv`): el nodo *a* debe dar aproximadamente 0,169 y todos los valores deben sumar 1.
- **Prueba con la muestra** (`aristas_muestra.tsv`) para confirmar que el formato de entrada del grafo es leído sin errores.
- **Ejecución completa** con 4 workers, dentro de screen:

```
hadoop jar mdp-giraph.jar PageRankRunner \
  /uhadoop2026/projects/grupo13/aristas.tsv /uhadoop2026/projects/grupo13/pr_dirigido 4
```

- **Ordenar el resultado** con `SortByRank -D mapreduce.job.reduces=1` y repetir con aristas_bidireccional.tsv si se quiere la variante B.
- **Comparar con Colab:** los valores se cruzan con `nodos.tsv` (nombres) y con las métricas de Pig, y se verifican contra `control_pagerank.csv`. Con la variante dirigida, el control de Colab da 142 proveedores en el perfil 1 y 10 en el perfil 2.